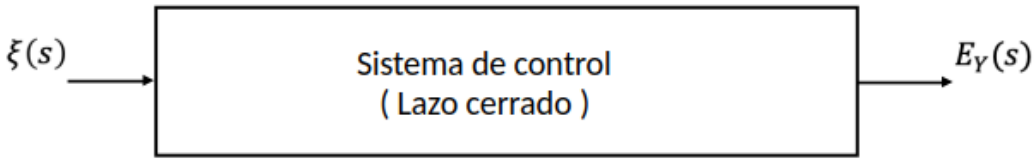
ADRC3

July 29, 2026

1 Active Disturbance Rejection Control

[11]:
```
import numpy as np
from sympy import *
from sympy import Matrix, symbols, init_printing
from sympy import Function, laplace_correspondence, diff, solve,symbols
import matplotlib.pyplot as plt
from control.matlab import *
from IPython.display import display, Latex
init_printing(use_latex='mathjax')
```

1.1 Observer-Based ADRC



1.1.1 Plant

First of all, we have to define our plant. The dynamic equation of the plant is:

$$\ddot{y} = u + \xi$$

where ξ is the disturbance of the system.

Now we are able to rewrite this in algebraic state-space form, which means separating the system into the plant system and the disturbance system.

Plant system:

$$\dot{X}_p = A_p X_p + B_p(u + \xi)$$

$$y = C_p X_p$$

Disturbance system:

$$\dot{X}_\xi = A_\xi X_\xi + B_\xi \cdot \xi^{(m)}$$

$$\xi = C_\xi X_\xi$$

Substituting ξ from the disturbance system into the plant system:

$$\dot{X}_p = A_p X_p + B_p(u + C_\xi X_\xi)$$

[12]:
```
s,kp = symbols('s kp')
k_p = 1
Gs=kp/(s**2)
display(Latex(r"G(s):"))
display(Gs)
num,den = fraction(Gs)
coeff = np.array(den.expand().as_poly(s).coeffs())
coeff
G_s = tf([k_p],[1,0,0])


sys_p = ss(G_s)
##state space Plant system
Ap = np.block([[0,1],[0,0]])
Bp = np.block([[0],[1]])
Cp = np.block([[1,0]])
Dp = np.block([[0]])
# Ap = sys_p.A
# Bp = sys_p.B
# Cp = sys_p.C
# Dp = sys_p.D
sys_p = ss(Ap,Bp,Cp,Dp)
display(Latex(r"State Space of Plant System:"))
display(sys_p)

#State Space disturbance system
Ad = np.block([[0,1,0],[0,0,1],[0,0,0]])
Bd = np.block([[0],[0],[0]])
Cd = np.block([[1,0,0]])
Dd = np.block([[0]])
```

```
sys_d = ss(Ad,Bd,Cd,Dd)
display(Latex(r"State Space of Disturbance System:"))
display(sys_d)
```

G(s):

$$\frac{kp}{s^2}$$

State Space of Plant System:

<StateSpace sys[11]: ['u[0]'] -> ['y[0]']>

$$\left[\begin{array}{cc|c} 0 & 1 & 0 \\ 0 & 0 & 1 \\ \hline 1 & 0 & 0 \end{array}\right]$$

State Space of Disturbance System:

<StateSpace sys[12]: ['u[0]'] -> ['y[0]']>

$$\left[\begin{array}{ccc|c} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 \end{array}\right]$$

1.1.2 Extended System

$$\begin{bmatrix} \dot{X}_p \\ \dot{X}_\xi \end{bmatrix} = \begin{bmatrix} A_p & B_p C_\xi \\ 0_{(m,n)} & A_\xi \end{bmatrix} \begin{bmatrix} X_p \\ X_\xi \end{bmatrix} + \begin{bmatrix} B_p \\ 0_{(m,1)} \end{bmatrix} u + \begin{bmatrix} 0_{(n,1)} \\ B_\xi \end{bmatrix} \xi^{(m)}$$

$$y = \begin{bmatrix} Cp & 0_{(1,m)} \end{bmatrix} \cdot \begin{bmatrix} X_p \\ X_\xi \end{bmatrix}$$

In simplified form:

$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot u + B_{\xi_{au}} \cdot \xi^{(m)}$$

$$C_{au} \cdot X_{au}$$

The state of augmented system are:

$$\dot{x_1} = x_2$$
$$\dot{x_2} = x_3 + u$$
$$\dot{x_3} = x_4$$
$$\dot{x_4} = x_5$$
$$\dot{x_5} = -\gamma_1 x_3 - \gamma_2 x_4 - \gamma_3 x_5 + PMI(\xi, t)$$

with the state vectors defined as:

$$X_p = \begin{bmatrix} y \\ \dot{y} \\ \ddot{y} \\ y^{(3)} \\ \vdots \\ y^{(n)} \end{bmatrix}, \qquad X_{\bar{\xi}} = \begin{bmatrix} \bar{\xi} \\ \dot{\bar{\xi}} \\ \ddot{\bar{\xi}} \\ \vdots \\ \bar{\xi}^{(m)} \end{bmatrix}$$

and the augmented state:

$$X_{au} = \begin{bmatrix} X_p \\ X_{\bar{\xi}} \end{bmatrix}$$

[13]:
```
n=Ap.shape[0]
m=Ad.shape[0]

Aau = np.block([[Ap,Bp@Cd],[np.zeros((m,n)),Ad]])
Bau = np.block([[Bp],[np.zeros((m,1))]])
Cau = np.block([Cp,np.zeros((1,m))])
Dau = np.block([[0]])
sys_au = ss(Aau,Bau,Cau,Dau)
display(Latex(r"State Space of Augmented System:"))
display(sys_au)
```

State Space of Augmented System:

<StateSpace sys[13]: ['u[0]'] -> ['y[0]']>

$$\left[\begin{array}{ccccc|c} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 1 & 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

1.1.3 Observer

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}u + Ly$$

with $y = C_p X_p$:

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}u + L \cdot C_{au}X_{au}$$

Now we define the estimation error as $\bar{e} = X_{au} - \widehat{X}_{au}$.

Estimation Error Dynamics

$$(\dot{X}_{au} - \dot{\widehat{X}}_{au}) = (A_{au} - LC_{au})(X_{au} - \widehat{X}_{au}) + B_{\xi_{au}}\xi^{(m)}$$

$$\dot{\bar{e}} = (A_{au} - LC_{au})\bar{e} + B_{\xi_{au}}\xi^{(m)}$$

If the disturbance is completely rejected by the differential operator, the estimation error dynamics converge to zero, indicating that the disturbance has been successfully rejected.

Laplace Form

$$\bar{e} \cdot (sI - (A_{au} - LC_{au})) = B_{\xi_{au}} s^m \xi$$

1.1.4 CloseLoop Extended system

the extended system but we are going to use the observer:

The system

$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot u + B_{\xi_{au}} \cdot \xi^{(m)}$$
$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot (r - K\widehat{X}) + B_{\xi_{au}} \cdot \xi^{(m)}$$

The observer

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}u + Ly$$
$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}(r - K\widehat{X}) + LC_{au}X_{au}$$

Extended system of the observer closeloop

$$\begin{bmatrix} \dot{X}_{au} \\ \dot{\widehat{X}}_{au} \end{bmatrix} = \begin{bmatrix} A_{au} & -B_{au}K \\ LC_{au} & (A_{au} - LC_{au}) - B_{au}K \end{bmatrix} \cdot \begin{bmatrix} X_{au} \\ \widehat{X}_{au} \end{bmatrix} + \begin{bmatrix} B_{au} \\ B_{au} \end{bmatrix} \cdot r$$

Estimation error representation

After that, it is necessary to define the estimation error: set $\bar{e} = X_{au} - \widehat{X}_{au}$

Now, we define the following transformation matrix, which allows us to change the system's representation into a triangular form. This new representation exposes the system's internal behavior more clearly, which in turn facilitates the computation of the gain vectors K and L.

$$P = \begin{bmatrix} I & 0 \\ I & -I \end{bmatrix}$$

$$\begin{bmatrix} X_{au} \\ \bar{e} \end{bmatrix} = \begin{bmatrix} I & 0 \\ I & -I \end{bmatrix} \cdot \begin{bmatrix} X_{au} \\ \widehat{X}_{au} \end{bmatrix}$$

extended system

$$\begin{bmatrix} \dot{X}_{au} \\ \dot{\bar{e}} \end{bmatrix} = \begin{bmatrix} (A_{au} - B_{au}K) & B_{au}K \\ 0_{(n+m,n+m)} & (A_{au} - LC_{au}) \end{bmatrix} \cdot \begin{bmatrix} X_{au} \\ \bar{e} \end{bmatrix} + \begin{bmatrix} B_{au} \\ 0_{(n+m,1)} \end{bmatrix} \cdot r$$

Design

equation 1: Control

$$det(SI - A_{au} + B_{au}K)$$

equation 2: Observer

$$det(SI - A_{au} + LC_{au})$$

[14]:
```
pols = np.array([
    -2,
    -4,
])
K_1= acker(Ap,Bp,pols).reshape(1,2)
K = np.block([K_1,1,np.zeros(2)]).reshape(1,5)
display(Latex(r"State Feedback Gain K:"))
display(K)

pols_L =3* np.array([
    -2,
    -4,
    -6,
    -8,
    -10,
])

L = acker(Aau.T,Cau.T, pols_L.T).reshape(5,1)
display(Latex(r"Observer Gain L:"))
display(L)
```

State Feedback Gain K:

```
array([[8., 6., 1., 0., 0.]])
```

Observer Gain L:

```
array([[9.00000e+01],
       [3.06000e+03],
       [4.86000e+04],
       [3.55104e+05],
       [9.33120e+05]])
```

1.1.5 Control law

The control law is given by the following expression, where the observer provides the estimated plant and disturbance states:

$$u = -k_1 \cdot \widehat{X}_{au_1} - k_2 \cdot \widehat{X}_{au_2} - k_3 \cdot \widehat{X}_{au_3} - \cdots - k_{n+m} \cdot \widehat{X}_{au_{n+m}}$$

$$u = K \cdot \widehat{X}_{au}$$

Linearization control

$$u = -K_p \widehat{X}_p - \hat{\xi}$$

$$u = -K \widehat{X}_{au}$$

1.1.6 Simulation system

First, to perform the simulation, it is important to create the closed-loop system including the observer. This closed-loop configuration is defined by the following system. **Plant:**

$$\dot{X}_p = A_p X_p + B_p u$$

with $u = r - K_{au} \widehat{X}_{au} + \xi$

$$\dot{X}_p = A_p X_p - B_p K_{au} \widehat{X}_{au} + B_p r + B_p \xi$$

Observer:

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au}) \widehat{X}_{au} + B_{au} u + Ly$$

with $y = C_p X_p$

and

$u = r - K_{au} \widehat{X}_{au}$

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au} - B_{au} K_{au}) \widehat{X}_{au} + B_{au} u + LC_p X_p$$

Closeloop System

$$\begin{bmatrix} \dot{X}_p \\ \dot{\widehat{X}}_{au} \end{bmatrix} = \begin{bmatrix} A_p & -B_p K \\ LC_p & (A_{au} - LC_{au} - B_{au} K) \end{bmatrix} \cdot \begin{bmatrix} X_p \\ \widehat{X}_{au} \end{bmatrix} + \begin{bmatrix} B_p & B_p \\ B_{au} & 0_{(n+m,1)} \end{bmatrix} \cdot \begin{bmatrix} r \\ \xi \end{bmatrix}$$

[15]:
```
## Close loop with Estended state observer and plant system:

A_cl = np.block([[Ap,-Bp@K],[L@Cp,Aau - L@Cau - Bau@K]])
B_cl = np.block([[Bp,Bp],[Bau+L,np.zeros((5,1))]])
C_cl = np.block([Cau,np.zeros((1,2))])
D_cl= np.block([np.zeros((1,2))])

print(A_cl.shape)
print(B_cl.shape)
print(C_cl.shape)
print(D_cl.shape)
sys_cl = ss(A_cl,B_cl,C_cl,D_cl)
display((sys_cl))
display(Latex(r"State Space of Extended System:"))
```

```
(7, 7)
(7, 2)
(1, 7)
(1, 2)
```

<StateSpace sys[14]: ['u[0]', 'u[1]'] -> ['y[0]']>

$$\left[\begin{array}{rrrrrrr|rr} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -8 & -6 & -1 & 0 & 0 & 1 & 1 \\ 90 & 0 & -90 & 1 & 0 & 0 & 0 & 90 & 0 \\ 3.06{\cdot}10^{3} & 0 & -3.07{\cdot}10^{3} & -6 & 0 & 0 & 0 & 3.06{\cdot}10^{3} & 0 \\ 4.86{\cdot}10^{4} & 0 & -4.86{\cdot}10^{4} & 0 & 0 & 1 & 0 & 4.86{\cdot}10^{4} & 0 \\ 3.55{\cdot}10^{5} & 0 & -3.55{\cdot}10^{5} & 0 & 0 & 0 & 1 & 3.55{\cdot}10^{5} & 0 \\ 9.33{\cdot}10^{5} & 0 & -9.33{\cdot}10^{5} & 0 & 0 & 0 & 0 & 9.33{\cdot}10^{5} & 0 \\ \hline 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

State Space of Extended System:

1.1.7 Simulation:

[16]:
```
t=np.linspace(0,10,1000)
r=np.ones_like(t)
xi=np.ones_like(t)*t**2
u = np.column_stack([r, xi])
xp0= np.array([0,0])
xp0_es= np.array([0,0])
xd0_es= np.array([1,0,0])

print(u.shape)
x0=  np.block([xp0,xp0_es,xd0_es])
[Y,T,X] = lsim(sys_cl, U=u, T=t, X0=x0)
print(X[-1,0])
print(X.shape)
Y1 = X[:,0]/(-0.87500)
```

```
(1000, 2)
-0.8750000003622522
(1000, 7)
```

1.1.8 PLOT

[17]:
```
fig, axs = plt.subplots(2,2, figsize=(12,8),layout='constrained')
axs[0,0].plot(T,X[:,0]/(-0.87500),color='red',␣
 ↪linewidth=1,linestyle='--',label='State x1:')
axs[0,0].plot(T,r,color='blue', linewidth=2,linestyle='-',label='Setpoint r')
axs[0,0].set_title('Graph of the output signal and setpoint')
axs[0,0].minorticks_on()
axs[0,0].legend()
axs[0,0].grid(visible = True,which='both')
```

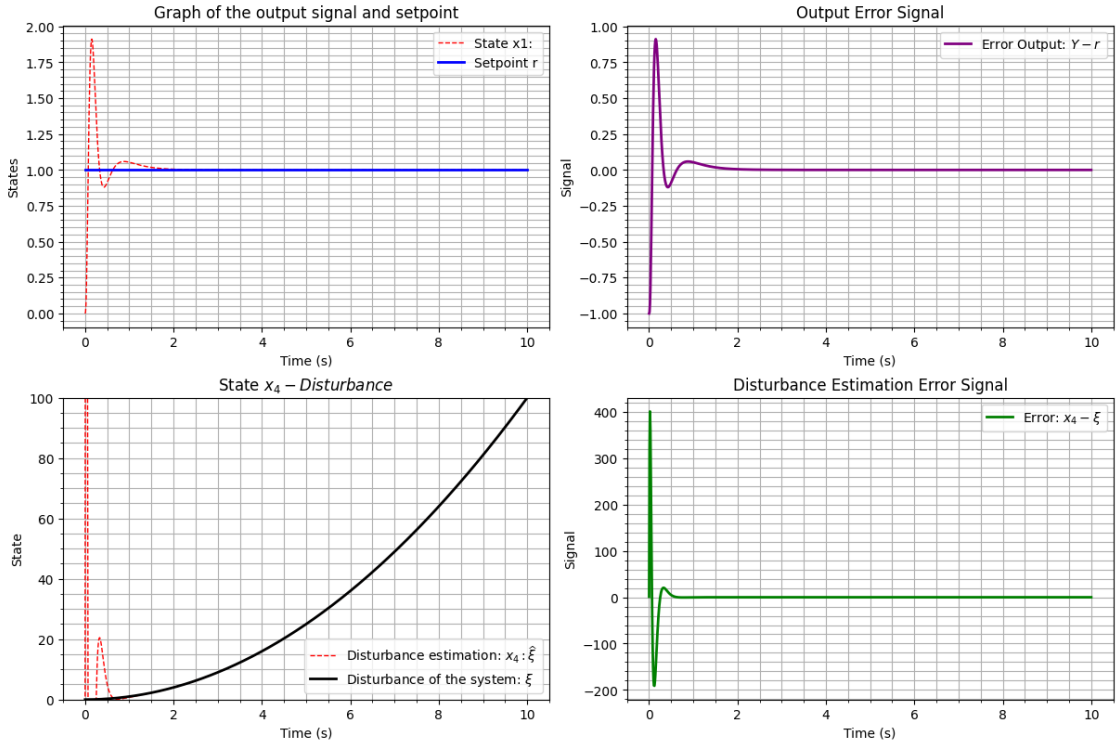
```
axs[0,0].set_xlabel('Time (s)')
axs[0,0].set_ylabel('States')

axs[1,0].plot(T,X[:,4],color='red',␣
 ↪linewidth=1,linestyle='--',label=r'Disturbance estimation: $x_{4}:
 ↪\widehat{\xi}$')
axs[1,0].plot(T,xi,color='black', linewidth=2,linestyle='-',label=r'Disturbance␣
 ↪of the system: $\xi$')
axs[1,0].set_title('State $x_{4}- Disturbance$')
axs[1,0].minorticks_on()
axs[1,0].legend()
axs[1,0].set_ylim(0,100)
axs[1,0].legend()
axs[1,0].grid(visible = True,which='both')
axs[1,0].set_xlabel('Time (s)')
axs[1,0].set_ylabel('State')

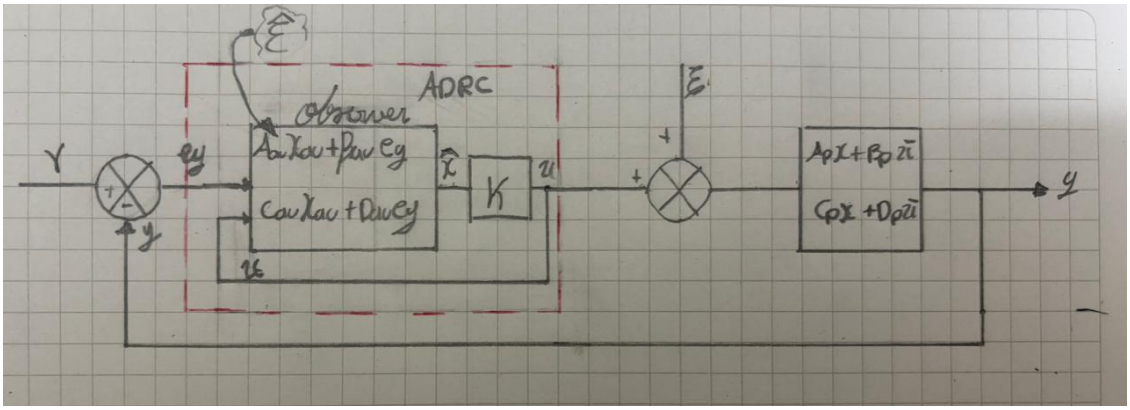
axs[1,1].plot(T,X[:,4]-xi,color='green',␣
 ↪linewidth=2,linestyle='-',label=r'Error: $x_{4}-\xi$')
axs[1,1].set_title('Disturbance Estimation Error Signal')
axs[1,1].minorticks_on()
axs[1,1].legend()
axs[1,1].grid(visible = True,which='both')
axs[1,1].set_xlabel('Time (s)')
axs[1,1].set_ylabel('Signal')

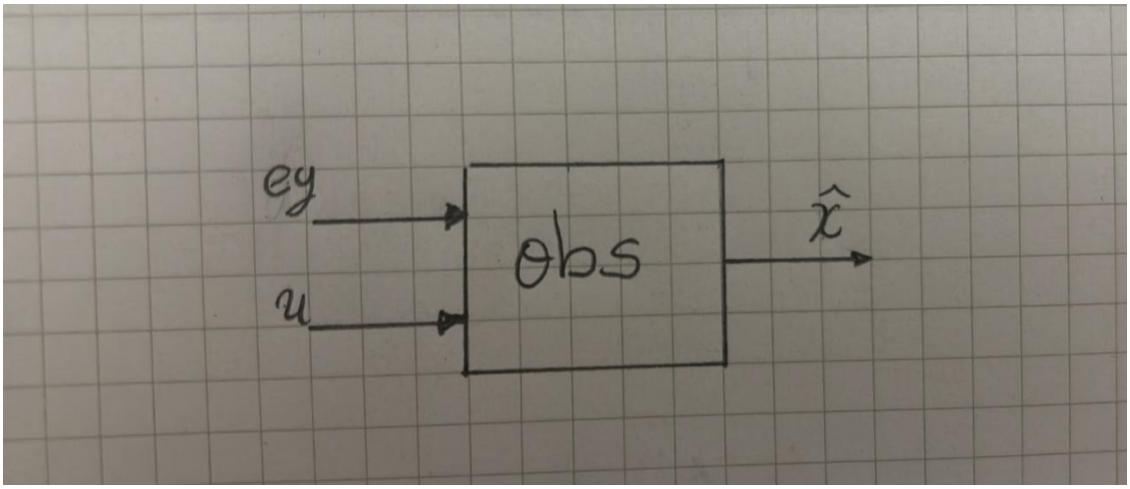

axs[0,1].plot(T,Y1-r,color='purple', linewidth=2,linestyle='-',label=r'Error␣
 ↪Output: $Y-r$')
axs[0,1].set_title('Output Error Signal')
axs[0,1].minorticks_on()
axs[0,1].legend()
axs[0,1].grid(visible = True,which='both')
axs[0,1].set_xlabel('Time (s)')
axs[0,1].set_ylabel('Signal')
```

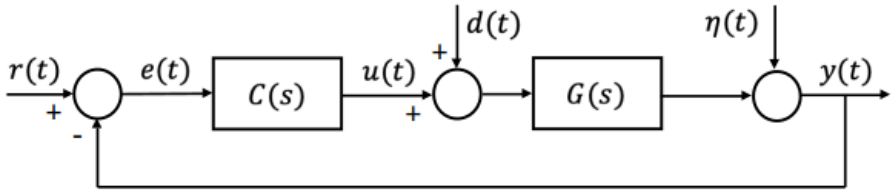
[17]: Text(0, 0.5, 'Signal')

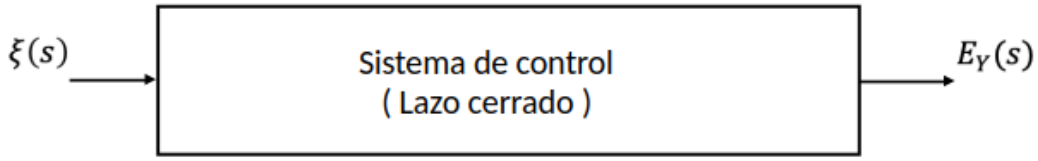


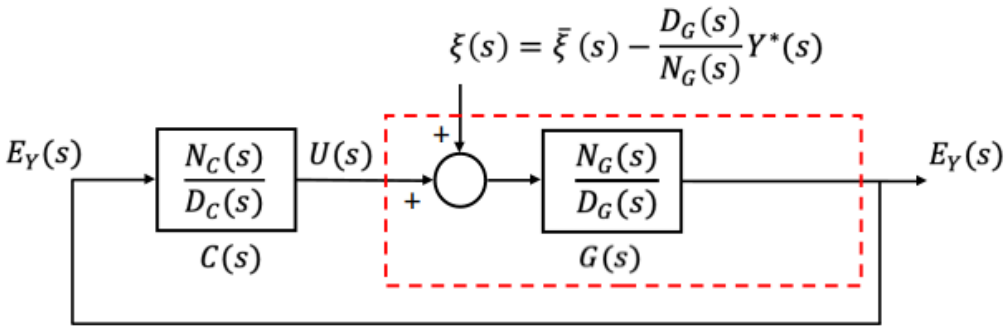
1.1.9 In Terms of the Tracking Error

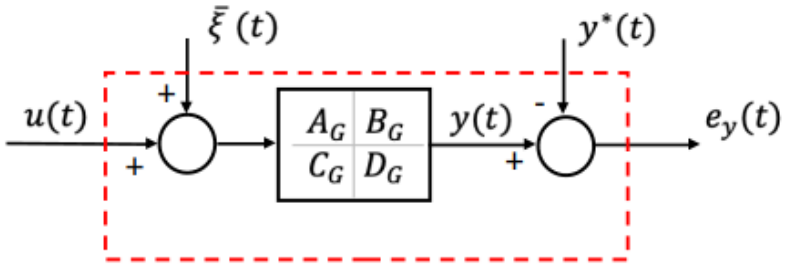



ey
u
obs
$\hat{x}$


r(t)
e(t)
C(s)
u(t)
d(t)
η(t)
G(s)
y(t)
+
-
+
+


ξ(s)
Sistema de control
(Lazo cerrado)
$E_Y(s)$


$\xi(s) = \bar{\xi}(s) - \frac{D_G(s)}{N_G(s)} Y^*(s)$
$E_Y(s)$
$\frac{N_C(s)}{D_C(s)}$
$C(s)$
$U(s)$
+
+
$\frac{N_G(s)}{D_G(s)}$
$G(s)$
$E_Y(s)$



$$\dot{x}_G(t) = A_G.x_G(t) + B_G.\left[u(t) + \bar{\xi}(t)\right]$$
$$y(t) = C_G.x_G(t) + D_G.u(t)$$

Calculation of u^* — Example with a Sample Plant

1. Constant Signals Assume constant reference signals:

$$y^*(t) = A, \qquad u^* = \text{constant}$$

Since the reference is constant:

$$\dot{y}^* = \ddot{y}^* = 0$$

Consider the following plant (used only for this example):

$$\frac{Y^*(s)}{U^*(s)} = \frac{2s-2}{s^2-s-6}$$

Multiplying both sides:

$$(s^2 - s - 6)y^* = (2s - 2)u^*$$

For constant signals, all derivatives are zero, so only the constant terms remain:

$$-6y^* = -2u^*$$

Therefore:

$$3y^* = u^*$$

If $y^*(t) = A$, then:

$$u^* = 3A$$

This means that the steady-state control signal must be **three times larger** than the reference amplitude.

---

2. Sinusoidal Signals Now consider a sinusoidal reference:

$$y^*(t) = A\sin(\omega t)$$

Assume a control signal of the form:

$$u^*(t) = a_1\cos(\omega t) + b_1\sin(\omega t)$$

Compute derivatives:

$$y^*(t) = A\sin(\omega t)$$

$$\dot{y}^*(t) = A\omega\cos(\omega t)$$

$$\ddot{y}^*(t) = -A\omega^2\sin(\omega t)$$

For the control signal:

$$u^*(t) = a_1\cos(\omega t) + b_1\sin(\omega t)$$

$$\dot{u}^*(t) = -a_1\omega\sin(\omega t) + b_1\omega\cos(\omega t)$$

Substitute into the differential equation:

$$(s^2 - s - 6)y^* = (2s - 2)u^*$$

Which in time domain becomes:

$$\ddot{y}^* - \dot{y}^* - 6y^* = 2\dot{u}^* - 2u^*$$

Substituting:

$$-A\omega^2\sin(\omega t) - A\omega\cos(\omega t) - 6A\sin(\omega t) = 2(-a_1\omega\sin(\omega t) + b_1\omega\cos(\omega t)) - 2(a_1\cos(\omega t) + b_1\sin(\omega t))$$

Separating sine and cosine terms leads to two equations:

$$-2\omega a_1 - 2b_1 = -A\omega^2 - 6A$$

$$-2a_1 + 2\omega b_1 = -A\omega$$

For:

$$\omega = 1, \qquad A = 1$$

The solution is:

$$a_1 = 2$$

$$b_1 = 1.5$$

Important Interpretation This example demonstrates that we **should not** simply invert the transfer function and multiply it by the reference:

$$u = G^{-1}(s)r$$

This approach can be dangerous. If the plant is **non-minimum phase** (i.e., it has a zero in the right half-plane), then inverting it would create an unstable pole. As a result, the controller would become unstable.

Instead, the correct interpretation is:

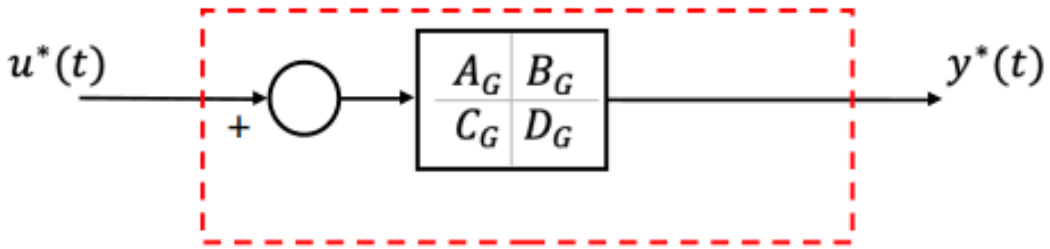
To achieve a desired output y^*, we must determine the appropriate control signal u^* that satisfies the plant dynamics.

In other words, the logic is inverted:

- We do not compute u by directly inverting the transfer function.
- We compute u^* by solving the differential equation that relates u^* and y^*.

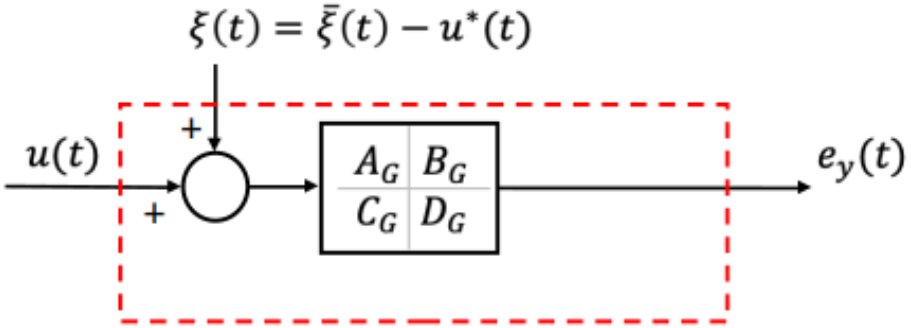
Therefore, if we want to impose a reference y^*, we must predict which control signal u^* makes that trajectory possible. This is obtained by solving the differential equation derived from the plant model.

Sistema Nominal NO perturbado



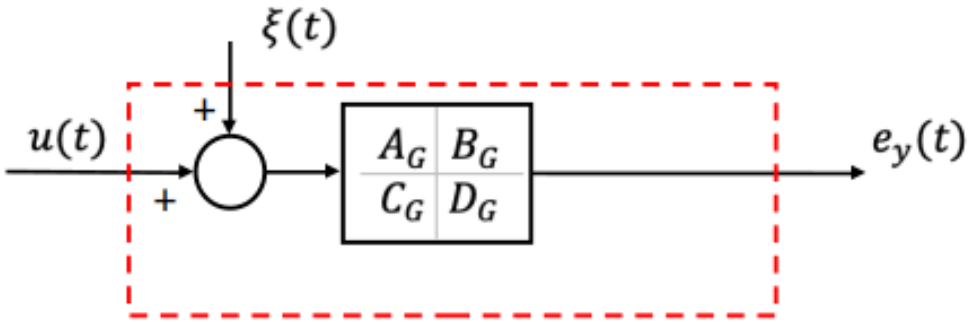
$$\dot{\tilde{x}}_G^*(t) = A_G.\tilde{x}_G^*(t) + B_G.u^*(t)$$

$$y^*(t) = C_G.\tilde{x}_G^*(t) + D_G.u^*(t)$$



$$\dot{\tilde{x}}_G(t) = A_G.\tilde{x}_G(t) + B_G.[u(t) + \xi(t)]$$

$$e_y(t) = C_G.\tilde{x}_G(t) + D_G.u(t)$$



$$\dot{\tilde{x}}_G(t) = A_G.\tilde{x}_G(t) + B_G.[u(t) + \xi(t)]$$
$$e_y(t) = C_G.\tilde{x}_G(t) + D_G.u(t)$$

1.1.10 Now, we have to design to the ADRC

We rewrite the second-order system as a second-order error system. Let $e_y = y - r$:

$$\ddot{e}_y = u + (\bar{\xi} - u^*)$$

Defining $\xi = \bar{\xi} - u^*$:

$$\ddot{e}_y = u + \xi$$

Applying the same procedure as before, the estimation error dynamics become: We have to select to the states of the system.

$$\dot{x_1} = x_2$$
$$\dot{x_2} = x_3 + u$$
$$\dot{x_3} = x_4$$
$$\dot{x_4} = x_5$$
$$\dot{x_5} = -\gamma_1 x_3 - \gamma_2 x_4 - \gamma_3 x_5 + PMI(\xi, t)$$

Extended Dynamics System

$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot u + B_{\xi_{au}} \cdot \xi^{(m)}$$

with the state vectors defined as:

$$X_p = \begin{bmatrix} e_y \\ \dot{e}_y \\ \ddot{e}_y \\ e_y^{(3)} \\ \vdots \\ e_y^{(n)} \end{bmatrix}, \qquad X_\xi = \begin{bmatrix} \xi \\ \dot{\xi} \\ \ddot{\xi} \\ \vdots \\ \xi^{(m)} \end{bmatrix}$$

and the augmented state:

$$X_{au} = \begin{bmatrix} X_p \\ X_\xi \end{bmatrix}$$

Observer System

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au}u + Le_y$$

with $e_y = C_{au} \cdot X_{au}$

Estimation Error Dynamics

$$(\dot{X}_{au} - \dot{\widehat{X}}_{au}) = (A_{au} - LC_{au})(X_{au} - \widehat{X}_{au}) + B_{\xi_{au}}\xi^{(m)}$$

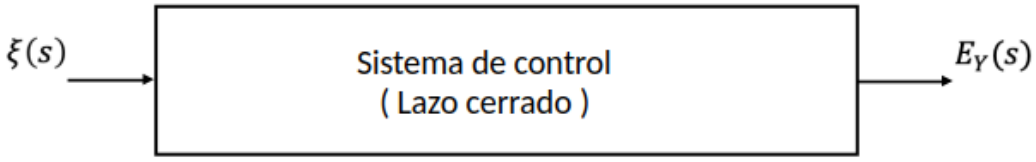
$$\dot{\bar{e}} = (A_{au} - LC_{au})\bar{e} + B_{\xi_{au}}\xi^{(m)}$$

The control law is given by the following expression, where the observer provides the estimated plant and disturbance states:

$$u = -k_1 \cdot X_{au_1} - k_2 \cdot X_{au_2} - k_3 \cdot X_{au_3} - \cdots - k_{n+m} \cdot X_{au_{n+m}}$$

$$u = -K\widehat{X}_{au}$$

1.1.11 The close loop system with the dinamc stimation error



The system is given for this equation: with $e_y = C_p X_p$,

$$u = -K \cdot \widehat{X}_{au}$$

and

$$\widehat{X}_{au} = X_{au} - \bar{e}$$

$$\widehat{X}_{au} = \begin{bmatrix} \widehat{X}_p \\ \widehat{X}_{\bar{\xi}} \end{bmatrix}$$

$$\dot{X}_{au} = A_{au} \cdot X_{au} + B_{au} \cdot u + B_{\xi_{au}} \cdot \xi^{(m)}$$

$$\dot{X}_{au} = A_{au} \cdot X_{au} - B_{au} K (X_{au} - \bar{e}) + B_{\xi_{au}} \cdot \xi^{(m)}$$

$$\dot{X}_{au} = (A_{au} + B_{au} K) \cdot X_{au} - B_{au} K \bar{e} + B_{\xi_{au}} \cdot \xi^{(m)}$$

Also, the observer:

$$\dot{\bar{e}} = (A_{au} - LC_{au})\bar{e} + B_{\xi_{au}} \xi^{(m)}$$

The extended system is:

$$\begin{bmatrix} \dot{X}_{au} \\ \bar{e} \end{bmatrix} = \begin{bmatrix} (A_{au} - B_{au}K) & B_{au}K \\ 0 & (A_{au} - LC_{au}) \end{bmatrix} \cdot \begin{bmatrix} X_{au} \\ \bar{e} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} \cdot u$$

1.1.12 Simulation system

First, to perform the simulation, it is important to create the closed-loop system including the observer. This closed-loop configuration is defined by the following system. **Plant:**

$$\dot{X}_p = A_p X_p + B_p \bar{u}$$

with $\bar{u} = u + \xi = -K_{au} \widehat{X}_{au} + \xi$

$$\dot{X}_p = A_p X_p - B_p K_{au} \widehat{X}_{au} + B_p \xi$$

Observer:

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} + B_{au} u + L e_y$$

with $ey = y - r$

and $u = -K_{au} \widehat{X}_{au}$; $y = C_p X_p$

$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au})\widehat{X}_{au} - B_{au}K_{au}\widehat{X}_{au} + L(y - r)$$
$$\dot{\widehat{X}}_{au} = (A_{au} - LC_{au} - B_{au}K_{au})\widehat{X}_{au} + LC_pX_p - Lr$$

Closeloop System

$$\begin{bmatrix} \dot{X}_p \\ \dot{\widehat{X}}_{au} \end{bmatrix} = \begin{bmatrix} A_p & -B_pK \\ LC_p & (A_{au} - LC_{au} - B_{au}K) \end{bmatrix} \cdot \begin{bmatrix} X_p \\ \widehat{X}_{au} \end{bmatrix} + \begin{bmatrix} 0_{(n,1)} & B_p \\ -L & 0_{(n+m,1)} \end{bmatrix} \cdot \begin{bmatrix} r \\ \xi \end{bmatrix}$$

[18]:
```
## Estended state observeri with the plant system:

Aau_e = np.block([[Ap,-Bp@K],[L@Cp,Aau - L@Cau - Bau@K]])
Bau_e = np.block([[np.zeros((2,1)),Bp],[-L,np.zeros((5,1))]])
Cau_e = np.block([Cau,np.zeros((1,2))])
Dau_e = np.block([np.zeros((1,2))])

print(Aau_e.shape)
print(Bau_e.shape)
print(Cau_e.shape)
print(Dau_e.shape)
sys_au_e = ss(Aau_e,Bau_e,Cau_e,Dau_e)
display((sys_au_e))
display(Latex(r"State Space of Extended System:"))
```

```
(7, 7)
(7, 2)
(1, 7)
(1, 2)
```

<StateSpace sys[15]: ['u[0]', 'u[1]'] -> ['y[0]']>

$$\left[\begin{array}{rrrrrrr|rr} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -8 & -6 & -1 & 0 & 0 & 0 & 1 \\ 90 & 0 & -90 & 1 & 0 & 0 & 0 & -90 & 0 \\ 3.06{\cdot}10^{3} & 0 & -3.07{\cdot}10^{3} & -6 & 0 & 0 & 0 & -3.06{\cdot}10^{3} & 0 \\ 4.86{\cdot}10^{4} & 0 & -4.86{\cdot}10^{4} & 0 & 0 & 1 & 0 & -4.86{\cdot}10^{4} & 0 \\ 3.55{\cdot}10^{5} & 0 & -3.55{\cdot}10^{5} & 0 & 0 & 0 & 1 & -3.55{\cdot}10^{5} & 0 \\ 9.33{\cdot}10^{5} & 0 & -9.33{\cdot}10^{5} & 0 & 0 & 0 & 0 & -9.33{\cdot}10^{5} & 0 \\ \hline 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

State Space of Extended System:

[19]:
```
t=np.linspace(0,10,1000)
r=np.ones_like(t)
xi=np.ones_like(t)*t**2
u = np.column_stack([r, xi])
xp0= np.array([0,0])
xp0_es= np.array([0,0])
```

```
xd0_es= np.array([1,0,0])

print(u.shape)
x0=  np.block([xp0,xp0_es,xd0_es])
[Y,T,X] = lsim(sys_au_e, U=u, T=t, X0=x0)
print(X[-1,0])
print(X.shape)
Y1 = X[:,0]
```

```
(1000, 2)
0.9999999998669685
(1000, 7)
```

[20]:
```
fig, axs = plt.subplots(2,2, figsize=(12,8),layout='constrained')
axs[0,0].plot(T,X[:,0],color='red', linewidth=1,linestyle='--',label='State x1:
 ↪')
axs[0,0].plot(T,r,color='blue', linewidth=2,linestyle='-',label='Setpoint r')
axs[0,0].set_title('Graph of the output signal and setpoint')
axs[0,0].minorticks_on()
axs[0,0].legend()
axs[0,0].grid(visible = True,which='both')
axs[0,0].set_xlabel('Time (s)')
axs[0,0].set_ylabel('States')

axs[1,0].plot(T,X[:,4],color='red',␣
 ↪linewidth=1,linestyle='--',label=r'Disturbance estimation: $x_{4}:
 ↪\widehat{\xi}$')
axs[1,0].plot(T,xi,color='black', linewidth=2,linestyle='-',label=r'Disturbance␣
 ↪of the system: $\xi$')
axs[1,0].set_title('State $x_{4}- Disturbance$')
axs[1,0].minorticks_on()
axs[1,0].legend()
axs[1,0].set_ylim(0,100)
axs[1,0].legend()
axs[1,0].grid(visible = True,which='both')
axs[1,0].set_xlabel('Time (s)')
axs[1,0].set_ylabel('State')

axs[1,1].plot(T,X[:,4]-xi,color='green',␣
 ↪linewidth=2,linestyle='-',label=r'Error: $x_{4}-\xi$')
axs[1,1].set_title('Disturbance Estimation Error Signal')
axs[1,1].minorticks_on()
axs[1,1].legend()
axs[1,1].grid(visible = True,which='both')
axs[1,1].set_xlabel('Time (s)')
axs[1,1].set_ylabel('Signal')
```

```
axs[0,1].plot(T,Y1-r,color='purple', linewidth=2,linestyle='-',label=r'Error␣
↪Output: $Y-r$')
axs[0,1].set_title('Output Error Signal')
axs[0,1].minorticks_on()
axs[0,1].legend()
axs[0,1].grid(visible = True,which='both')
axs[0,1].set_xlabel('Time (s)')
axs[0,1].set_ylabel('Signal')
```

[20]: Text(0, 0.5, 'Signal')

